

```

    }
  }
  @media (min-width: 1280px) {
    width: 800px;
  }
}

outputs:
.component {
  width: 300px;
}
@media (min-width: 768px) {
  .component {
    width: 600px;
  }
}
@media (min-width: 768px) and (min-
resolution: 192dpi) {
  .component {
    background-image: url(/img/
retina2x.png);
  }
}
@media (min-width: 1280px) {
  .component {
    width: 800px;
  }
}

```

Operations

Arithmetical operations such as +, -, *, /, can operate on any number, colour or variable. If it is possible, mathematical operations take units into account and convert numbers before adding, subtracting or comparing them. The result has the leftmost explicitly stated unit type. If the conversion is impossible or not meaningful, units are ignored. Examples of impossible conversion are px to cm or rad to %.

```

// numbers are converted into the same
units
@conversion-1: 5cm + 10mm; // result
is 6cm
@conversion-2: 2 - 3cm - 5mm; // result
is -1.5cm

// conversion is impossible
@incompatible-units: 2 + 5px - 3cm; //
result is 4px

```

```

// example with variables
@base: 5%;
@filler: @base * 2; // result is 10%
@other: @base + @filler; // result is 15%

```

Multiplication and division do not convert numbers. It would not be meaningful in most cases — a length multiplied by a length gives an area, and CSS does not support specifying areas. Less operates on numbers as they are, and assigns explicitly stated unit type to the result:

```
@base: 2cm * 3mm; // result is 6cm
```

You can also do arithmetic on colours with Less:

```

@color: #224488 / 2; //results in
#112244
background-color: #112244 + #111; //
result is #223355

```

Functions

Less provides a variety of functions that transform colours, manipulate strings and do maths. Using them is pretty straightforward. The following example uses percentage to convert 0.5 to 50 per cent, increases the saturation of a base colour by 5 per cent, and then sets the background colour to one that is lightened by 25 per cent and spun by 8 degrees:

```

@base: #f04615;
@width: 0.5;

.class {
  width: percentage(@width); // returns
`50%`
  color: saturate(@base, 5%);
  background-color: spin(lighten(@base,
25%), 8);
}

Maps
As of Less 3.5, you can also use mixins
and rulesets as maps of values.
#colors() {
  primary: blue;
  secondary: green;
}

```

```

.button {
  color: #colors[primary];
  border: 1px solid #colors[secondary];
}

```

As expected, this gives the output:

```

.button {
  color: blue;
  border: 1px solid green;
}

```

Sass and Less

Less is inspired by Sass. The latter was designed to both simplify and extend CSS, so things like curly braces were removed from the syntax. Less was designed to be as close to CSS as possible, and as a result existing CSS can be used as valid Less code.

To assign variables, Sass uses '\$' while Less uses '@'. The problem here is that there are a few existing CSS selectors that already use '@'. At times, this can be a little confusing. Sass and Less have extensions available to integrate mixins (the ability to store and share CSS declarations throughout a site). Sass uses Compass for mixins, which includes every option available with updates for future support. Less has *Preboot*, *less*, *Less Mixins*, *Less Elements*, *gs*, and *Frameless*. Software support for these extensions is supposedly quite fragmented, which means that we may have to use a combination of these.

Sass and Less were both initially on Ruby. Less was then ported to JavaScript. Much later, Ruby Sass was moved to Dart Sass, which compiles to pure JavaScript. **END** 

References

- [1] <http://lesscss.org/>
- [2] [https://en.wikipedia.org/wiki/Less_\(stylesheet_language\)#Less_software](https://en.wikipedia.org/wiki/Less_(stylesheet_language)#Less_software)

 By: Athira Lekshmi

The author is an open source enthusiast.